MPOO 2027-1 · GRUPO 06

Parcial 1 · correcciones

Versión B del examen del 7 de octubre. Para cada pregunta vienen el enunciado, tu respuesta tal como la enviaste en el portal y su corrección. La calificación no va aquí: la ves con el profesor.

Antes de empezar. Este archivo lo subió el profesor a tu rama. Trae el cambio a tu computadora antes de tu siguiente push, o tu push va a fallar:

```
git checkout entregas_leon_diego
git pull --no-rebase
```

Qué tienes que hacer. En hojas, **a mano**, copia cada pregunta y escribe debajo tu respuesta corregida, usando la corrección que viene aquí. Donde la corrección ya te da la salida o el código, no basta con copiarlo: escribe **por qué** sale así, con la tabla de seguimiento de las variables, la causa del error o lo que devuelve tu método en cada renglón de la tabla. Debajo de esas preguntas viene qué escribir. Escanea las hojas o tómales foto, júntalas en un PDF y súbelo a esta misma carpeta, `entregas/leon_diego/examen/p1/`, con tu push de siempre.

Pregunta 0.1 · Preguntas abiertas

Pregunta

Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de código
En la radiografía de un objeto, ¿qué representa un atributo y qué representa un método? Da un ejemplo de cada uno.

Tu respuesta

```
Un atributo es una especificación que se le va a dar a un objeto, un metodo en cambio es  un medio por el cual se tendra que imprimir el objeto
```

Corrección. Ver el atributo como una 'especificación' del objeto se acerca a la idea de estado, pero decir que un método es 'un medio para imprimir el objeto' es un error conceptual, y faltan los ejemplos. Lo correcto: el atributo guarda parte del estado (por ejemplo, la velocidad de un coche) y el método expresa un comportamiento (por ejemplo, acelerar).

Pregunta 0.2 · Preguntas abiertas

Pregunta

¿Qué hace el método main y por qué se declara static en los programas que usamos en clase?

Tu respuesta

```
main se encarga de inicializar variables, porque no modifica el objeto
```

Corrección. Main no sirve para inicializar variables, y 'no modifica el objeto' no explica por qué es static. main es el punto de entrada donde arranca la ejecución; es static para poder llamarlo desde la clase sin crear antes una instancia.

Pregunta 0.3 · Preguntas abiertas

Pregunta

¿Qué diferencia hay entre un atributo de instancia y uno static? Explica qué comparten dos objetos de la misma clase.

Tu respuesta

```
Uno de instancia es uno que puede cambiar el estado del objeto segun ciertos requisitos se pidan para llegar al cambio, mientras que los static son atributos que se conservaran sin importar lo que se introdusca en la terminal
```

Corrección. Ligar el atributo de instancia al estado del objeto es una idea válida. Pero static no significa que el valor se quede igual pase lo que pase (eso se parece más a final), y la respuesta no dice qué comparten los objetos. Cada objeto tiene su propia copia del atributo de instancia, y el static pertenece a la clase y es un solo dato compartido por todas las instancias, como un contador de objetos creados.

Pregunta 0.4 · Preguntas abiertas

Pregunta

¿Qué guarda una variable de tipo primitivo y qué guarda una de referencia? Da un ejemplo de cada una.

Tu respuesta

```
Primitivo:

Referencia:
```

Corrección. Sin respuesta.

Pregunta 0.5 · Preguntas abiertas

Pregunta

¿Qué hace javac con un archivo .java y qué hace la JVM con el resultado?

Tu respuesta

```
La terminal crea un archivo para el .java de .class y aparte crea un nuevo archivo con los resultados del JVM
```

Corrección. Reconoce que del .java sale un .class, aunque se lo atribuye a 'la terminal' y no al compilador javac. La JVM no crea un archivo de resultados. Lo correcto: javac compila el fuente a bytecode (.class) y la JVM ejecuta ese bytecode.

R1 · ¿Qué imprime?

Pregunta

Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido.

```
int[] datos = {7, 5, 3};
int[] alias = datos;
```

```
int i = 0;
while (i < 3) {
    System.out.println(alias[i] / 2);
    i++;
}
alias = new int[]{9, 8, 6};
System.out.println("B" + datos[0] + alias[0]);
double mitad = (double) (datos[1] / 2);
System.out.println(mitad);
```

Tu respuesta

```
Va a imprimir
3.5, 2.5, 1.5
B +
() (/2)
```

Corrección. La salida correcta era 3 / 2 / 1 / B79 / 2.0. En las tres primeras líneas sí identificas que se divide cada elemento entre 2, pero usas división real en lugar de entera (int/int trunca). En la cuarta línea solo es correcta la 'B': faltó concatenar datos[0]=7, que no cambia al reasignar alias, y alias[0]=9. La quinta línea no tiene un valor reconocible: el cast ocurre después de 5/2=2, así que imprime 2.0.

Qué escribir a mano. La salida ya la tienes en la corrección. Escribe por qué sale así: una tabla de seguimiento con el valor de cada variable en cada vuelta y en cada println.

R2 a) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
int modo = 1;
switch (modo) {
    case 1: System.out.println("LISTO");
    case 2: System.out.println("FIN");
}
```

Tu respuesta

Clasificación

```
Compila pero truena
```

Causa o salida

```
FIN //El switch solo esta igualada a una sola respuesta que es 1, asi como que no da
instrucciones claras para llegar al caso 1 ni para llegar al caso 2
```

Corrección. La clasificación es incorrecta: el fragmento funciona. FIN, una de las dos líneas de la salida, sí es correcta. La salida completa es LISTO / FIN, porque sin break la ejecución sigue del case 1 al case 2 (fall-through).

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 b) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
{ int nivel = 4; }
System.out.println(nivel);
```

Tu respuesta

Clasificación

```
Funciona
```

Causa o salida

```
4
```

Corrección. Incorrecto: no compila. nivel solo existe dentro de las llaves donde se declara, así que fuera de ese bloque no está definida y el println no compila.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 c) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
String etiqueta = null;
System.out.println(etiqueta.trim());
```

Tu respuesta

Clasificación

```
No compila
```

Causa o salida

```
Se pide imprimir una etiqueta.trim cuando originalmente solo se tiene un String etiqueta
```

Corrección. La clasificación es incorrecta y la causa no menciona null. Sí compila (etiqueta es un String válido), pero truena en ejecución porque trim() se llama sobre una referencia null y no hay objeto (NullPointerException).

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R3 · Modelado: base, reto y justificación

Pregunta

Cada cargador fija su capacidad al fabricarse y nace vacío. Cuenta ranuras ocupadas y cargadores fabricados. Escribe colocar() y una consulta de ocupadas. Supón que solo se llama colocar cuando hay espacio. No uses arreglos.

Base. Declara al menos dos atributos de instancia con tipo, uno static y una constante final (¿también static?). Completa constructor y dos métodos: cambio y consulta.

Reto. ¿Qué es final y qué static: capacidad o fabricados? ¿Alguno necesita ambos?

Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

Tu respuesta

Clase

```
public class porcentaje () {
    static porcentaje = 0
return CARGAR
}

public class bateria () {
    static bateria == porcentaje
return Cargar
}
```

Justificaciones

```
iniliciacion de la carga en 0 para saber si es necesario cargar una bateria
```

> **Corrección.** No modela el cargador. Escribe dos 'clases' (porcentaje y bateria) con atributos static sin tipo y un 'return CARGAR' que viene del R5. No hay capacidad, ranuras ocupadas, contador de fabricados, constructor, colocar() ni consulta, y el reto queda sin respuesta. Un modelo esperado: final int capacidad e int ocupadas de instancia, static int fabricados = 0, un constructor que asigna this.capacidad e incrementa fabricados, colocar() { ocupadas++; } y una consulta que devuelve ocupadas; capacidad es final y fabricados es static, y ninguno necesita ambos.

R4 · Código con repetición

Pregunta

El arreglo intentos contiene exactamente tres códigos enteros, en el orden en que se probaron. Devuelve el número del primer intento que coincide con clave (1, 2 o 3). Si ninguno coincide, devuelve 0. Usa do-while y deja de revisar al acertar. El arreglo no es null. No lo modifiques.

```
static int intentoValido(int[] intentos, int clave)
```

Ejemplos (argumentos → retorno):

{13, 42, 8}, 42 → 2 · {42, 8, 9}, 42 → 1 · {1, 2, 3}, 42 → 0

Escribe el método completo. No imprimas ni leas del teclado.

Tu respuesta

```
static int intentoValido(int [] intentos, int clave)
switch (intentos)
    caso 1: intentos //es igual al argumento al segundo intento
        return 2
    caso 2: intentos == argumento //es igual al argumento al primer intento
        retutn 1
    caso 3: intentos //es diferente de argumento
        return 0

//Entendi que se tienen que tener un resultado por cada caso
```

Corrección. La cabecera es correcta. Usa un switch sobre el arreglo en lugar de do-while y nunca accede a intentos[i]; solo se ve la intención de comparar con la clave. En límites se reconoce que acertar en el primer intento devuelve 1 y que no acertar devuelve 0. Resultado: tiene un return por caso y no imprime ni modifica el arreglo. La sintaxis es pseudocódigo ('caso', 'retutn', sin llaves): 0. Lo esperado era recorrer con do-while un índice i de 0 a 2, regresar i+1 en el primer intentos[i]==clave y regresar 0 al terminar.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.

R5 · Código con selección

Pregunta

Devuelve la acción del módulo con switch sobre Modo. Primero: si bateria < 10, devuelve "CARGAR", incluso con modo null. En otro caso, null devuelve "SIN MODO" antes del switch. El enum ya existe: escribe solo el método. No imprimas ni leas del teclado.

```
enum Modo { APAGADO, MEDIR, CALIBRAR, PRUEBA }
static String accion(Modo modo, int bateria)
```

Regla o valor recibido	Texto devuelto
APAGADO	"DETENER"
MEDIR	"LEER"
CALIBRAR	"AJUSTAR"
PRUEBA	"SIMULAR"

Tu respuesta

```
public class bateria() {
    static String accion(Modo modo, int bateria)
    enum Modo { APAGADO, MEDIR, CALIBRAR, PRUEBA }
        Switch (Modo) {
            caso 1: ( bateria < 10)
                p
            caso 2: (
    }
}
//Por medio de un switch se busca saber que se hace si una bateria ya esta cargada o no, pero
para eso tiene que primero leer el porcentaje de la misma para posteriormente decidir cual
caso tomar
```

Corrección. La cabecera es correcta y elige un switch como estructura. La respuesta quedó incompleta: la condición bateria < 10 aparece dentro de un case y no antes del switch, y no devuelve "CARGAR". Faltan el caso null con "SIN MODO", las cuatro correspondencias (DETENER, LEER, AJUSTAR, SIMULAR) y los return. Lo esperado: if (bateria < 10) return "CARGAR"; if (modo == null) return "SIN MODO"; y luego switch (modo) con un return por cada constante.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.